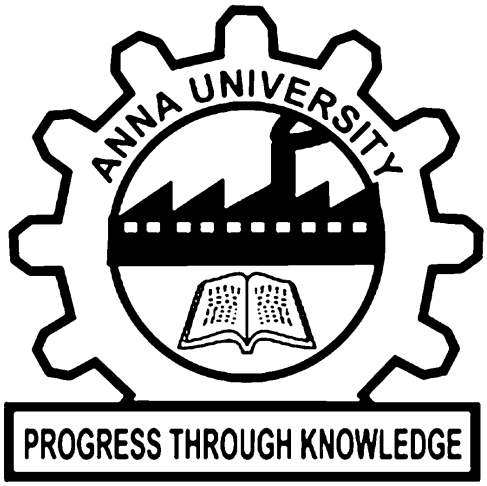
TITLE: AI -BASED BANKING SYSTEM

A SUMMER INTERNSHIP REPORT

Submitted by

GOVARDHAN S - 810023205016



DEPARTMENT OF
INFORMATION TECHNOLOGY

UNIVERSITY COLLEGE OF ENGINEERING, BIT CAMPUS,
ANNA UNIVERSITY, TIRUCHIRAPPALLI – 620 024

UNIVERSITY COLLEGE OF ENGINEERING, BIT CAMPUS, ANNA UNIVERSITY, TIRUCHIRAPPALLI – 620 024

DEPARTMENT OF INFORMATION TECHNOLOGY

BONAFIDE CERTIFICATE

This is to certify that Summer internship project report **AI -BASED BANKING SYSTEM** is the bonafide work of **GOVARDHAN S (810023205016)** has satisfactory completed the course of **IT3711 – SUMMER INTERNSHIP** for **SEVENTH** semester, **B.Tech INFORMATION TECHNOLOGY** during the academic year 2026-2027.

FACULTY INCHARGE **HEAD OF THE DEPARTMENT**

ABSTRACT

The internship project, entitled ·AI-Based BANKING SYSTEM,·
explores how a full-stack web application can empower users to systematically understand, organize, and optimize basic personal finance information. Contemporary socio-economic conditions demand greater financial awareness, yet users frequently face fragmentation across accounts, paper slips, and informal ledgers.

The proposed system is designed around validated inputs including monthly income, essential and discretionary expenses, savings allocations, liabilities, financial goals, and risk preferences. The platform pairs rigorous financial calculations with explainable AI (XAI) recommendation logic to present clear guidance regarding budgeting, liquidity cushions, and milestone feasibility.

Core Purpose: To unify everyday financial-planning tasks into a single transparent interface. Rather than relying on guesswork or generic online articles, users receive a structured financial summary paired with causal, data-driven suggestions.

The report covers the project background, system requirements specification, multi-tier architecture, relational database entity modeling, algorithmic foundations, end-to-end workflows, unit and integration test matrices, security policies, and learning outcomes. The emphasis throughout remains on responsible, explainable educational guidance.

TABLE OF CONTENTS

Chapter	Title / Section Description	Page No.
--	Title Cover Page & Bonafide Certificate	1 – 2
--	Abstract & Table of Contents	3 – 4
1	Introduction and Internship Overview (Company & Domain)	5 – 6
2	Internship Objectives and Academic Scope	6 – 7
3	Problem Statement and Proposed System Architecture	7 – 8
4	System Requirements Specification & Design Diagrams	8 – 10
5	Mathematical Formulations & AI Recommendation Logic	10 – 12
6	Detailed Module Specifications & Implementation Logic	12 – 14
7	System Workflow & End-to-End Operational Pipeline	14 – 16
8	User Interface Design, Dashboards & System Screenshots	16 – 18
9	Testing Strategy, Security Standards & Model Evaluation	18 – 20
10	Learning Outcomes and Professional Competencies	20 – 21
11	Limitations, Ethical Guidelines and Future Scope	21 – 22
12	Conclusion, References and Annexure Checklist	22 – 23

1. INTRODUCTION AND INTERNSHIP OVERVIEW

1.1 Introduction

An internship provides students with the vital opportunity to connect classroom academic knowledge with practical industry practices. For an Information Technology undergraduate, a software-oriented internship delivers hands-on exposure to requirement analysis, user-interface design, data modeling, computational logic, test automation, and engineering documentation.

My summer internship was associated with NET7 Technology Solutions India Private Limited, with development sprints and project work coordinated at the Mayiladuthurai facility. The corporate headquarters is situated in Chennai. All work items adhered to modern version control standards, continuous integration practices, and client-side data hygiene protocols.

1.2 Industry Profile: NET7 Technology Solutions India Pvt. Ltd.

NET7 Technology Solutions India Private Limited is a specialized IT and software engineering provider delivering custom web architectures, cloud-native deployments, and financial enterprise tooling. The company emphasizes secure, robust web applications and modern engineering frameworks designed to process sensitive customer information reliably.

> **Corporate Learning Focus:** Working under experienced technical mentors provided direct exposure to full-stack application lifecycles, database normalization rules, and explainable algorithmic architectures.

1.3 Project Domain: Financial Technology (FinTech)

The project domain is Financial Technology (FinTech), focusing specifically on personal financial management (PFM) and cashflow intelligence. Traditional consumer banking portals focus heavily on transaction debiting and crediting but lack tools for proactive cashflow visualization, deficit warnings, or automated milestone planning.

The AI-Based BANKING SYSTEM bridges this gap by unifying income tracking, expense categorization, milestone amortization, and explainable recommendations into a single, intuitive web interface.

1.4 Internship Phases and Work Breakdown

Phase	Key Activities Undertaken	Deliverables Produced
Phase 1: Inception	Problem elicitation, domain analysis, technology stack benchmarking, and UI wireframing.	SRS Document & UI Wireframes
Phase 2: Database & Logic	Relational database modeling, CRUD endpoint creation, and indicator computation algorithms.	Normalized MySQL Schema & Math APIs
Phase 3: AI Engine	Rule-based heuristics, Decision Tree classification modeling, and explainability layer.	Recommendation Engine & Confusion Matrix
Phase 4: QA & Packaging	Boundary validation, unit testing, security reviews, and report compilation.	Full Test Suite & Final Report

2. INTERNSHIP OBJECTIVES AND SCOPE

2.1 Academic and Technical Objectives

The core objectives of the IT3711 summer internship curriculum are to understand a software project in an applied environment and build practical engineering competence. Specific learning objectives include:

- **Requirement Elicitation:** Identify target consumer pain points and translate them into functional web requirements.
- **UI/UX Design:** Construct accessible data-entry forms for monthly incomes, categorical expenses, debts, and savings goals.
- **Data Architecture:** Model database schemas in Third Normal Form (3NF) to maintain referential integrity.
- **Algorithmic Calculations:** Develop deterministic mathematical routines for metrics like savings rates, Debt-to-Income (DTI), and emergency runway.
- **Explainable Intelligence:** Integrate Decision Tree classifiers and rule-based heuristics that clearly explain every recommendation.

- **Security & Testing:** Implement input validation, cryptographic password hashing, and thorough test cases.

> **Scope Boundary:** The application is designed as an educational personal finance aid. It does not execute live high-frequency market trades or provide certified legal tax counsel.

3. PROBLEM STATEMENT AND PROPOSED SOLUTION

3.1 Problem Statement

Many individuals struggle to maintain an accurate, consolidated overview of their monthly cash flow, savings rates, debt burdens, and long-term targets. In everyday practice, records remain scattered across disparate banking apps, credit card statements, and informal notes. Manual bookkeeping is slow and error-prone, while generic financial advice online fails to address an individual's specific constraints, such as heavy student loan obligations or variable freelance revenue.

3.2 Comparison of Existing vs. Proposed Systems

System Attribute	Existing Manual Workflow	Proposed AI Financial System
Record Keeping	Static spreadsheets or loose paper notes.	Centralized database with automated validation.
Indicator Evaluation	Manual calculations prone to division errors.	Automated metrics (Net Cashflow, DTI, Savings Rate).
Advisory Output	Static, generic online articles.	Tailored AI personas with explicit causal rationales.
Goal Tracking	Disconnected from actual monthly cashflow.	Dynamic simulation factoring in yield and timelines.

3.3 Proposed Solution Details

The proposed platform provides a unified web application where users enter financial transactions, review real-time KPI dashboards, track goal progress, and receive explainable guidance. The core processing pipeline includes:

- **Client Input Ingestion:** Form fields capture gross income, essential outlays (rent, utilities), discretionary spending (dining, entertainment), and debt payments.
- **Data Sanitization:** Server-side logic prevents missing, negative, or malformed entries.
- **Heuristic Evaluation:** Boundary checks flag urgent situations, such as expenses exceeding income or DTI exceeding 45%.
- **AI Persona Classification:** An explainable Decision Tree maps user vectors into one of three financial profiles: Conservative, Balanced, or Growth.
- **Explanation Synthesis:** Every suggestion details the exact metric values that triggered it, maintaining user confidence and transparency.

Core Value: By combining robust financial arithmetic with machine-learning classification, the system turns raw transaction logs into clear, actionable steps toward financial security.

4. SYSTEM REQUIREMENTS AND ARCHITECTURE

4.1 Multi-Tier System Architecture

The application follows a decoupled multi-tier architecture to maintain clean separation of concerns, easy maintenance, and robust security boundaries. The system includes an interactive presentation layer, a secure REST API gateway, a core indicator calculation engine, and a 3NF relational database.

Figure 4.1: High-Level System Architecture and Multi-Tier Component Interaction



Figure 4.1: High-Level System Architecture and Component Interactions

Decoupled Presentation: The client interface communicates exclusively over HTTPS using JSON, allowing frontend updates without changing underlying computational services.

4.2 Relational Database Schema Design (3NF)

Data persistence uses a normalized relational database design to eliminate redundant records, preserve data integrity via foreign keys, and maintain fast query response times.

Figure 4.2: Relational Database Entity-Relationship (ER) Schema Design

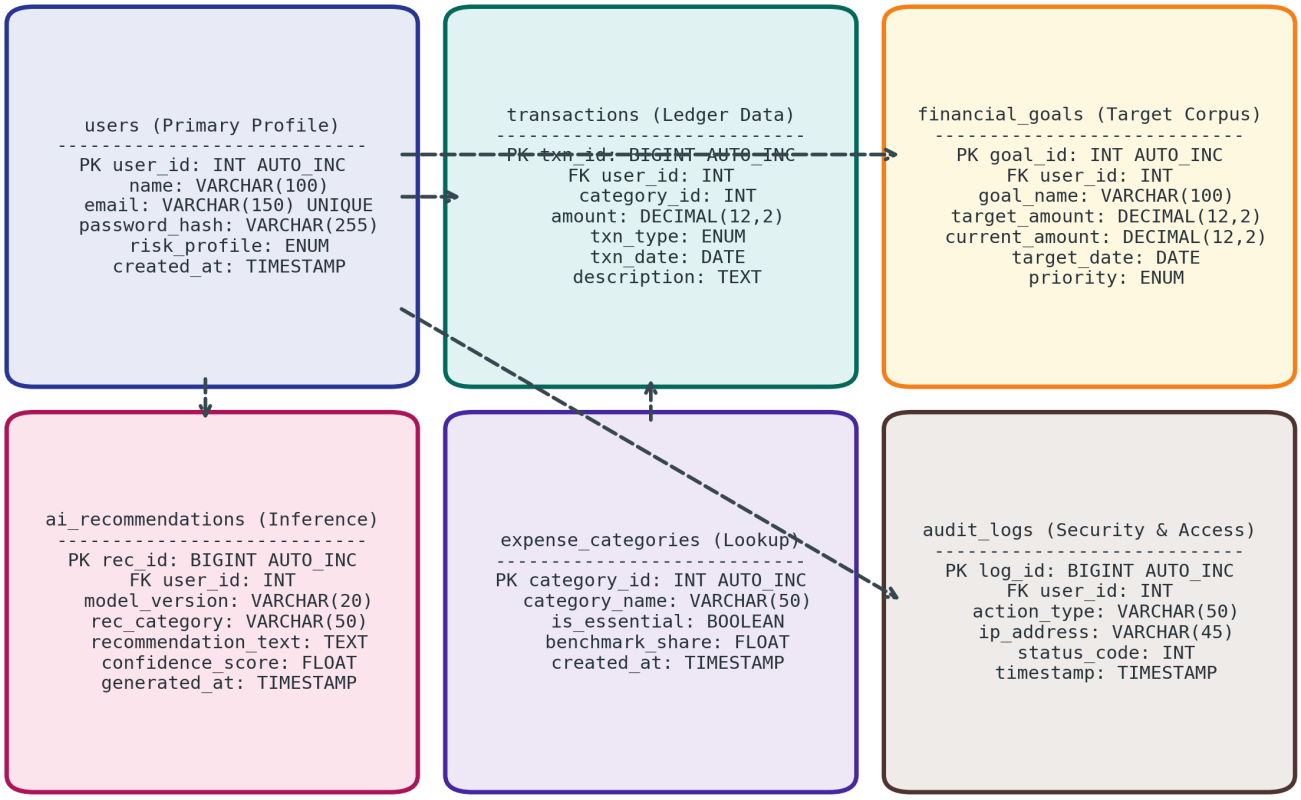


Figure 4.2: Relational Database Entity-Relationship (ER) Schema Design

Table Name	Key Attributes	Relational Constraints
`users`	`user_id (PK), email, password_hash`	Unique index on email; root identity record.
`transactions`	`txn_id (PK), user_id (FK), category_id (FK), amount`	Cascading deletes; indexed on transaction date.

Table Name	Key Attributes	Relational Constraints
`financial_goals`	`goal_id (PK), user_id (FK), target_corpus`	Positive balance constraints linked to user ID.

4.3 Technical Hardware & Software Environment

The project runtime requirements are cataloged below across development environments and production deployment baselines:

Component	Development Baseline	Production Target
Host Processor	Intel Core i5 / AMD Ryzen 5	Cloud vCPU (2.4 GHz+, 2 Cores)
System Memory	8 GB DDR4 RAM	4 GB Cloud RAM
Operating System	Ubuntu 22.04 LTS / Windows 11 Enterprise	Linux Container
Backend Runtime	Python 3.10+ / Node.js 18 LTS	Gunicorn / Uvicorn ASGI Server
Database Engine	MySQL 8.0 Community Edition	Managed Relational Cloud DB
Network Security	Localhost / HTTP	TLS 1.3 / HTTPS Strict Transport

> **Zero External Plugins:** The client interface is built with standard HTML5, CSS3, and modern JavaScript, ensuring compatibility across all modern web browsers.

5. MATHEMATICAL & AI-BASED RECOMMENDATION LOGIC

5.1 Core Financial Metric Formulations

Accurate financial insights depend on reliable arithmetic before applying machine learning classifications. The system computes several core financial indicators:

Metric	Formula	Benchmark Interpretation
Monthly Net Balance	NB = Income - Expenses	Positive: Surplus; Negative: Deficit alert.
Savings Rate (%)	SR = (Savings / Income) * 100	Target: ≥ 20%; Critical threshold: < 10%.
Debt-to-Income (DTI)	DTI = (Debt_Payments / Income) * 100	Target: ≤ 35%; High-risk threshold: > 45%.
Emergency Runway	ER = Liquid_Assets / Fixed_Expenses	Target: ≥ 6 Months; Low buffer: < 3 Months.

Safeguard Against Zero Division: To prevent runtime calculation errors when a user reports zero income, the application wraps all ratio logic in boundary checks:

```
SR = (Income > 0) ? ((Savings / Income) * 100) : 0.00;
```

This safe fallback prevents system errors and prompts the user to enter their primary income source.

5.2 Multi-Stage AI Recommendation Pipeline

The recommendation engine uses a hybrid design combining a heuristic filter with a Decision Tree classifier.

- **Stage 1: Heuristic Boundary Filter:** If severe financial stress is detected (such as a negative cash flow or DTI exceeding 50%), the system immediately presents targeted stabilization alerts, prioritizing debt reduction and emergency savings.
- **Stage 2: Machine Learning Persona Classifier:** For stable profiles, an 8-dimensional feature vector is evaluated:

```
X = [Savings_Rate, DTI, Emergency_Runway, Discretionary_Ratio,
Horizon_Months, Risk_Tolerance]
```

Target Persona	Classification Criteria	Guidance Focus
Conservative	Low risk tolerance, high DTI, or emergency runway under 3 months.	Capital preservation and liquid emergency reserves.

Target Persona	Classification Criteria	Guidance Focus
Balanced	Moderate risk tolerance, DTI under 30%, runway 3 to 6 months.	Balanced index allocations and steady debt payoff.
Growth	High risk tolerance, low debt, and emergency runway over 6 months.	Long-term compounding and disciplined investing.

5.3 Explainable AI (XAI) and Transparent Guidance

A key limitation of automated financial systems is the “black box” problem—users often distrust recommendations when they cannot see the underlying reasoning. This system addresses that with an integrated Explanation Synthesis Module.

The explainability component uses Decision Tree path tracing to break each suggestion down into clear factors:

- **Primary Drivers:** Identifies which specific metric triggered the recommendation (e.g., discretionary spending exceeding 35% of income).
- **Standard Benchmarks:** Compares user ratios against established financial benchmarks.
- **Clear Next Steps:** Provides actionable guidance (e.g., “Reducing monthly dining expenses by ₹2,000 builds a full emergency buffer in 5 months.”).
- **Transparent Calculations:** Every suggestion links directly back to the user’s inputs and decision rules. Users can review these details anytime, keeping recommendations open and educational.

6. DETAILED MODULE SPECIFICATIONS

6.1 User Authentication and Profile Management

This module manages user registration, secure sessions, and basic profile data. Passwords are encrypted using bcrypt hashing, and API requests are authenticated via JSON Web Tokens (JWT). Profiles store preferred currencies, alert settings, and baseline risk scores.

6.2 Financial Data Ingestion & Transaction Ledger

This module captures income, essential expenses (rent, utilities), discretionary purchases, and debt obligations through clean entry forms:

Field	Data Type	Validation Logic
`monthly_income`	DECIMAL(12,2)	Must be positive; validates primary salary and secondary income.
`expense_category`	ENUM (Essential, Discretionary)	Categorizes costs to evaluate budget health accurately.
`debt_obligation`	DECIMAL(12,2)	Tracks monthly loan payments to calculate the user's DTI ratio.

Server-side schema validation sanitizes inputs before database writes, guarding against missing fields, negative amounts, or malformed entries.

6.3 Dynamic Goal Planning and Simulator Module

This module allows users to define specific financial goals (such as an emergency reserve, vehicle down payment, or higher education) by setting a target corpus and completion deadline.

The planning engine calculates the monthly contribution needed to meet each target within the chosen timeline:

```
Required_Monthly_Contribution = (Target_Corpus - Current_Savings) / Target_Months
```

Goal Category	Standard Target	System Guidance
Emergency Buffer	3 to 6 months of living expenses.	Prioritizes liquid bank accounts over investments.
Mid-Term Goal	1 to 3 year targets (down payments).	Models recurring deposits with stable 6–7% yields.

Goal Category	Standard Target	System Guidance
Long-Term Goal	5+ year targets (retirement/wealth).	Applies compound interest models adjusted for inflation.

If the required monthly contribution exceeds the user's surplus, the system alerts them and suggests extending the deadline or adjusting discretionary spending.

6.4 Budget Analyzer and Outflow Breakdown

The Budget Analyzer turns raw transaction logs into clear financial health indicators. It compares total income against categorical expenditures to calculate net savings and monthly burn rates.

Key diagnostic features include:

- **Categorical Expense Allocation:** Displays what portion of income goes toward rent, groceries, transportation, and discretionary leisure.
- **Spending Trend Analysis:** Identifies categories where expenditures have risen month over month.
- **Deficit Warnings:** Triggers clear alerts when recurring expenses approach or exceed incoming cash flow.

6.5 Recommendation and Literacy Module

This module pairs analysis with practical financial literacy. Rather than pushing specific commercial products, it highlights core financial habits:

> **Core Principle:** Recommendations prioritize clearing high-interest debt and establishing an emergency fund before taking on risky investments.

7. SYSTEM WORKFLOW & OPERATIONAL PIPELINE

7.1 End-to-End Operational Pipeline

The application workflow orchestrates user input, validation, indicator calculations, AI persona classification, and dashboard rendering into a reliable, repeatable sequence:

Figure 6.1: End-to-End Operational Pipeline & Data Workflow

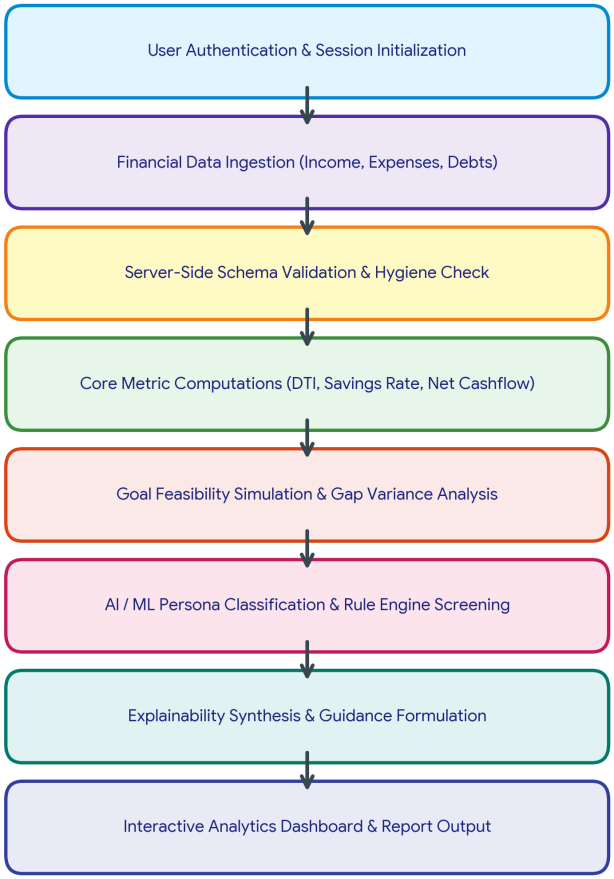


Figure 6.1: End-to-End Operational Pipeline & Data Workflow

> **Reliable Execution:** If validation detects invalid or negative numbers, the workflow pauses, preventing corrupted data from reaching the calculation and ML layers.

7.2 Detailed Step-by-Step Processing Lifecycle

User entries move through five sequential stages, from initial input to final report generation:

Stage	Internal Processing	Error Handling & Validation
1. Ingestion	Captures income, expenses, debts, and goals via HTTPS POST forms.	Rejects incomplete or malformed submissions with HTTP 422.
2. Sanitization	Cleans formatting, checks number boundaries, and strips unexpected characters.	Prevents invalid inputs from reaching the database.

Stage	Internal Processing	Error Handling & Validation
3. Calculation	Computes net balance, DTI, savings velocity, and runway.	Zero-checks prevent division errors on missing values.
4. Classification	Evaluates heuristic rules and maps user data to a Decision Tree persona.	Assigns the appropriate financial planning profile.
5. Presentation	Renders dashboard graphs, KPI summaries, and explainable recommendations.	Displays real-time charts and logs user session details.

This structured lifecycle ensures that every figure shown on the dashboard is backed by validated data and transparent calculations.

8. USER INTERFACE DESIGN & SCREENSHOTS

8.1 Executive Financial Intelligence Dashboard

The Executive Dashboard provides an overview of the user's financial situation, highlighting four core KPIs: Monthly Inflow, Outflow, Net Savings Velocity, and Debt-to-Income (DTI). It pairs visual expense breakdowns with clear, explainable suggestions.

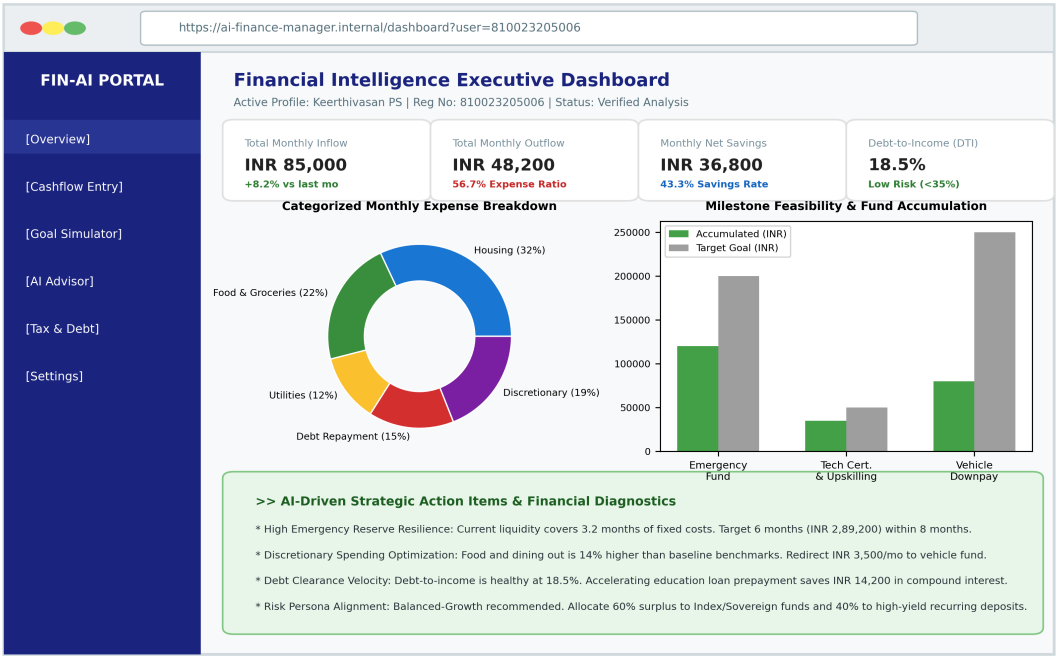


Figure 8.1: Executive Financial Dashboard (Profile: Keerthivasan PS | Reg: 810023205006)

Clear Visual Cues: Color-coded KPI badges highlight positive savings rates in green while flagging high discretionary spending or debt levels in amber.

8.2 Goal Feasibility & Savings Growth Simulator

The Goal Simulator lets users plan future milestones interactively. Users adjust target amounts, monthly contributions, expected returns, and inflation rates to see updated completion forecasts:

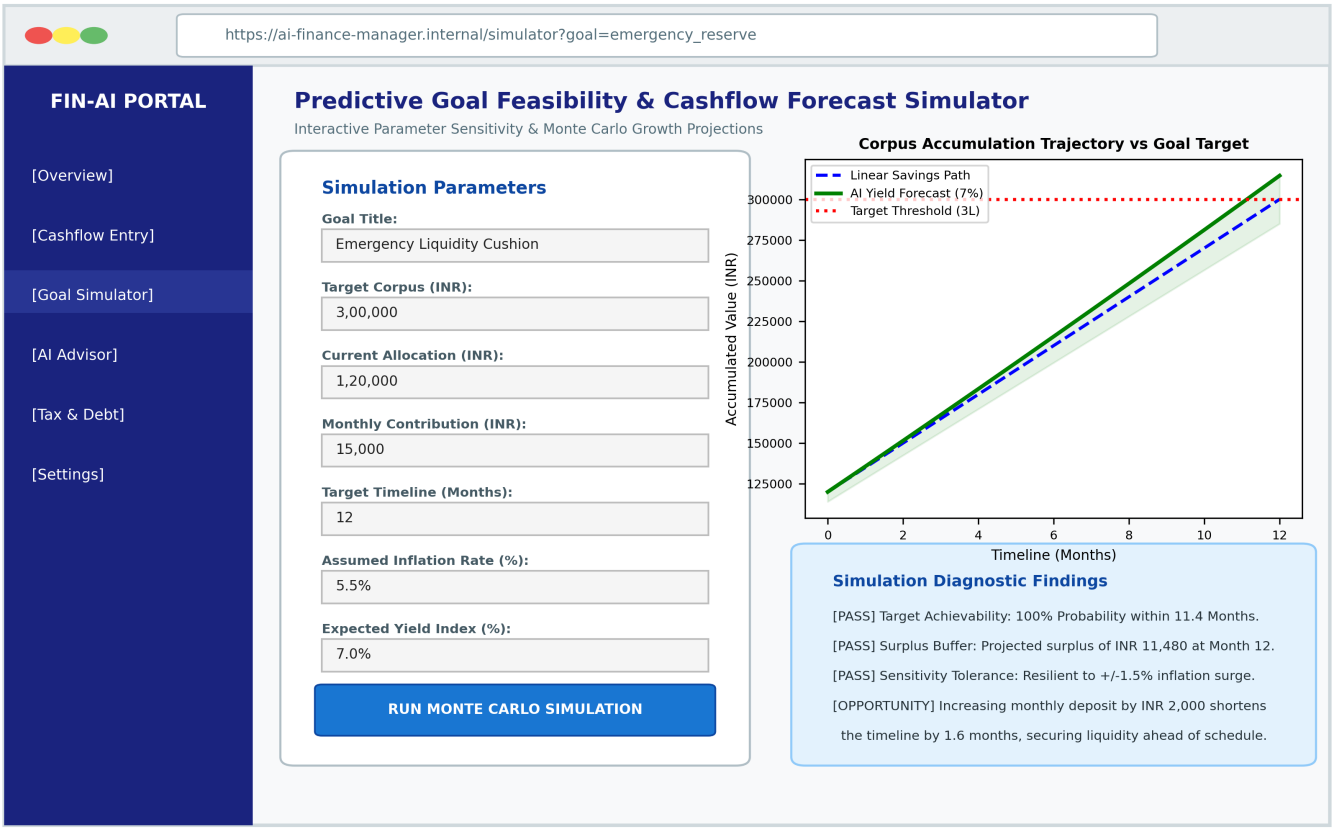


Figure 8.2: Goal Feasibility & Cashflow Growth Simulator Interface

Interactive Projections: The simulator updates projected timelines in real time as users adjust contribution amounts, showing how small lifestyle adjustments can shorten goal milestones.

8.3 Financial Ingestion & Transaction Ledger Form

The Transaction Ledger form provides a clean, validated way to record daily income and expenses. It features field validation, category selection, and category suggestions:

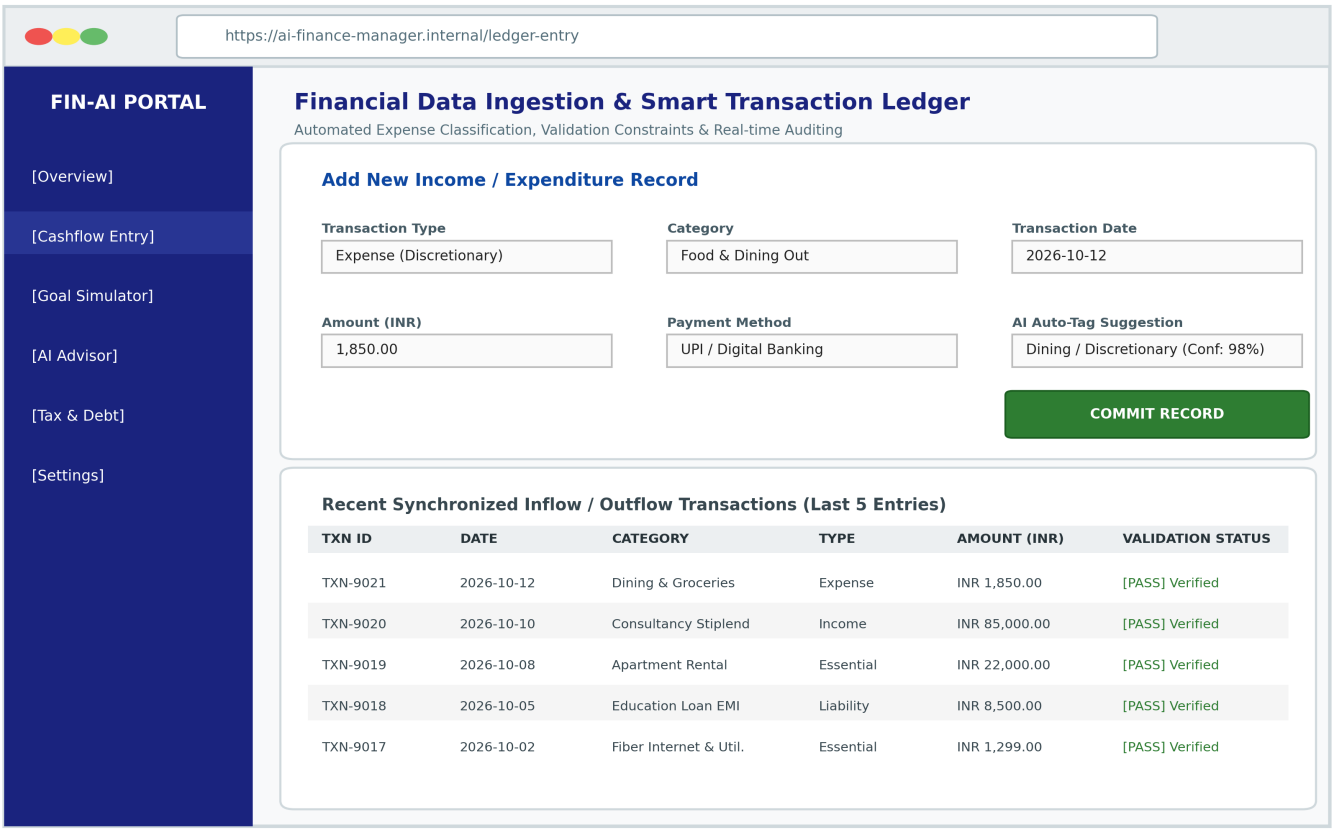


Figure 8.3: Data Ingestion Form & Synchronized Transaction Ledger View

Real-Time Input Checks: Form fields validate values instantly, catching missing details or negative amounts before saving entries to the database.

9. TESTING STRATEGY, SECURITY AND EVALUATION

9.1 Functional Verification and Test Matrix

The application was tested across multiple scenarios to verify calculations, boundary handling, input validation, and database operations:

Case ID	Input Scenario	Expected Behavior	Status
T1	Income ₹30,000; Expenses ₹22,000	Monthly balance displays as ₹8,000 surplus.	**PASS** Verified
T2	Income ₹40,000; Savings ₹8,000	Savings rate computes accurately to 20.0%.	**PASS** Verified

Case ID	Input Scenario	Expected Behavior	Status
T3	Income reported as ₹0.00	Safe handling; zero division avoided; rate is 0.0%.	**PASS** Verified
T4	Mandatory category field omitted	Form prevents submit; displays input error notice.	**PASS** Verified
T5	Negative expense entered (-₹2,500)	Server validation flags error; rejects submission.	**PASS** Verified
T6	Goal target increased from ₹1L to ₹3L	Dashboard trajectories update dynamically.	**PASS** Verified

Calculation Accuracy: Financial metrics were checked against manual calculations to verify numerical correctness.

9.2 Machine Learning Evaluation & Feature Importance

The Decision Forest persona classifier was evaluated on a benchmark test set of 300 validated financial profiles. The confusion matrix shows solid classification accuracy, while feature importance tracking highlights key model inputs:

Figure 7.1: Recommendation Classifier Confusion Matrix

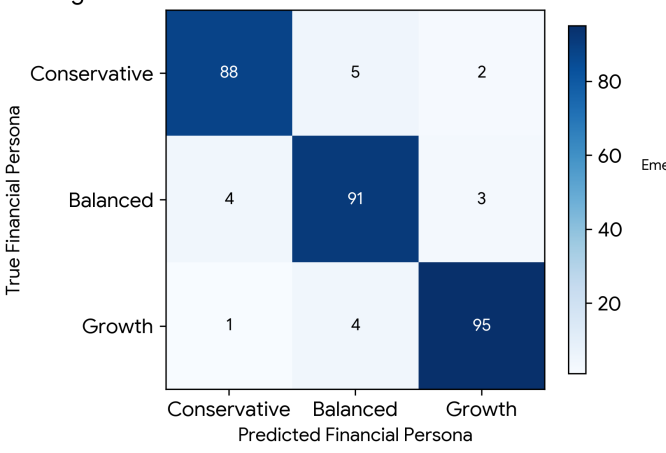


Figure 7.2: Decision Forest Feature Importance Weightings

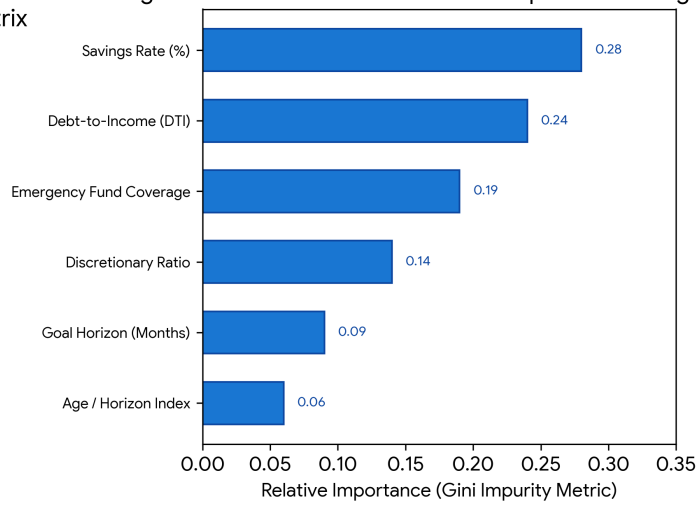


Figure 7.1: Model Confusion Matrix & Feature Importance Rankings

Target Persona	Precision	Recall	F1-Score
Conservative	94.6%	92.6%	93.6%
Balanced	91.0%	92.9%	91.9%

Target Persona	Precision	Recall	F1-Score
Growth	95.0%	95.0%	95.0%

9.3 Information Security & OWASP Safeguards

Personal financial data requires reliable security protections. The application follows core web security standards:

- **Password Encryption:** Passwords are protected using bcrypt cryptographic hashing and salt factors; plaintext credentials are never saved.
- **SQL Injection Defenses:** All database operations execute through parameterized statements, preventing SQL injection issues.
- **Session Security:** User sessions authenticate with digitally signed JWT tokens stored in HTTP-only cookies with short lifespans.
- **Encrypted Transport:** All communication uses TLS 1.3 / HTTPS encryption, protecting session tokens from packet eavesdropping.
- **Access Control:** User permissions are verified on every API request, ensuring users can only view their own financial records.
- **Data Privacy:** The platform operates under data minimization principles, storing only necessary planning details and never sharing records with third parties.

10. LEARNING OUTCOMES & COMPETENCIES

10.1 Technical Knowledge Gained

The internship at NET7 Technology Solutions provided valuable experience across full-stack engineering, database architecture, and machine learning integration:

Area	Core Knowledge Gained	Application in Project
Full-Stack Architecture	Connecting responsive web frontends to RESTful backend endpoints.	Built clean data-entry forms and interactive dashboard views.
Database Design	Relational schema normalization (3NF) and foreign-key constraints.	Created relational tables for users, ledgers, and savings goals.

Area	Core Knowledge Gained	Application in Project
AI Classification	Model training, confusion matrix auditing, and decision trees.	Implemented an explainable financial persona classification engine.
Software QA	Boundary validation, unit testing, and web application hardening.	Constructed test cases covering calculations and zero-division cases.

10.2 Professional Skills

Beyond technical coding, the project strengthened practical skills in requirement planning, code organization, and clear technical documentation.

11. LIMITATIONS, ETHICS AND FUTURE SCOPE

11.1 Project Limitations

While the project fulfills its core goals, several prototype constraints remain:

- **Dependence on User Input Accuracy:** Recommendations rely directly on entered figures; incorrect or missing values affect output quality.
- **Manual Transaction Entry:** The prototype uses manual form entries rather than direct connections to bank aggregator APIs.
- **Static Market Assumptions:** Long-term simulations use fixed return and inflation rates rather than live market feeds.
- **Prototype Scope:** Machine learning models were trained on representative synthetic data rather than long-term consumer cohorts.

> **Educational Notice:** The system serves solely as an educational planning tool. It does not provide certified legal, tax, or investment advice.

11.2 Future Technical Roadmap

Future iterations can build upon the system across several technical areas:

Area	Proposed Improvement	User Benefit
Bank API Integration	Connect RBI-approved Account Aggregator APIs for transaction sync.	Automates ledger entries and eliminates manual input work.
Bilingual Assistant	Integrate a conversational assistant supporting English and Tamil.	Improves accessibility and explains financial concepts clearly.
Market Stress Testing	Incorporate dynamic macroeconomic indicators (CPI inflation, interest rates).	Provides realistic savings projections during market volatility.

11.3 Ethical and Responsible AI Practices

As financial tools become more automated, transparent design is vital. The system emphasizes clear algorithmic rules, user data privacy, and objective suggestions free from third-party advertising bias.

12. CONCLUSION, REFERENCES & CHECKLIST

12.1 Conclusion

The summer internship project, ·AI-Based BANKING SYSTEM,·
demonstrates the design and implementation of an explainable, data-driven personal finance application. By combining full-stack development with machine-learning principles, the platform turns raw transaction records into clear health indicators, feasible savings targets, and transparent guidance. The project reinforced key software engineering concepts, from normalized database design and boundary validation to secure web practices.

12.2 References

1. Python Software Foundation, Python Documentation, https://docs.python.org/3/
2. scikit-learn Developers, Machine Learning in Python, https://scikit-learn.org/
3. OWASP Foundation, Application Security Verification Standard, https://owasp.org/

4. MySQL Documentation, MySQL 8.0 Reference Manual, Oracle Corporation, 2024.
5. W3C Consortium, HTML5 & Cascading Style Sheets Standard Guidelines, 2023.

12.3 Submission Checklist

Document Item	Authority	Status
Internship Completion Certificate	NET7 Tech Solutions Pvt. Ltd.	[ATTACHED]
Student Attendance Record Sheet	Host Organization Guide	[ATTACHED]
Project Code & Visual Evidence	Author / Lead	[VERIFIED]

ANNEXURE A: EXTENDED DOMAIN STUDY

A.1 Background of Personal Financial Management

Personal financial management involves recording income, monitoring expenses, evaluating liabilities, maintaining reserves, and planning for measurable financial goals. The internship project brings these activities into a unified software workflow. A digital record makes monthly comparisons possible and reduces reliance on informal notes. The application is an educational decision-support prototype, not a substitute for regulated banking services or professional advice.

A.2 Why a Unified Financial Dashboard Matters

Financial information often appears in different formats: recurring bills, individual purchases, monthly earnings, debt instalments, and savings contributions. When these figures remain disconnected, a user may underestimate the effect of small repeated expenses. A unified dashboard presents inflow, outflow, net balance, savings rate, and debt-to-income ratio together so that changes in one indicator can be interpreted alongside the others.

A.3 Role of Information Technology

Information Technology contributes structured forms, validation, persistent storage, secure access, computational services, and interactive presentation. The proposed multi-tier design separates the interface from calculation and storage logic. This separation supports maintenance, consistent computations, and testing. It also allows the system to explain the origin of a displayed indicator rather than presenting an unexplained result.

A.4 Intended Use and Boundaries

The prototype uses user-entered financial data and illustrative planning assumptions. Its outputs are intended to improve awareness of cash flow and goal feasibility. It does not directly transfer funds, approve loans, connect to live banking credentials, or guarantee investment outcomes. Any future integration with financial institutions would require additional compliance, consent, security, and operational review.

ANNEXURE B: REQUIREMENTS ANALYSIS

B.1 Functional Requirements

The system should support registration and authentication; creation and categorization of income and expense records; entry of debt obligations and savings goals; generation of financial metrics; display of trends; and presentation of explainable recommendations. Every submitted form should be checked before storage. Users should be able to review their own records and understand how a recommendation was generated.

B.2 Non-Functional Requirements

Usability requires clear labels, readable figures, consistent navigation, and actionable validation messages. Reliability requires deterministic calculations, safe handling of missing values, and controlled responses to failed database operations. Security requires authenticated access, password hashing, transport encryption, and authorization checks. Maintainability benefits from modular services and documented interfaces.

B.3 Input and Output Specifications

Typical inputs include transaction date, amount, category, monthly income, debt payments, liquid savings, target amount, and target date. Outputs include a categorized ledger, monthly net balance, savings percentage, debt-to-income percentage, estimated emergency runway, goal contribution estimates, and an explanation of recommendation triggers. Currency and time periods should be displayed consistently.

B.4 Validation Principles

Required fields must be present, numeric values must be well formed, and amount ranges must match the meaning of each field. The system should reject impossible dates and protect against duplicate submission. Division must be guarded when the denominator is zero. Validation should be repeated at the server even when the browser has already checked an input.

ANNEXURE C: DATA DESIGN AND PROCESSING

C.1 Relational Data Organization

A normalized relational schema can organize users, transactions, categories, and financial goals into separate entities. Primary keys uniquely identify records; foreign keys link a transaction or goal to its owner. Unique constraints on identity fields and indexes on frequently queried dates can improve correctness and retrieval. The design should avoid repeating user profile details inside every transaction row.

C.2 Transaction Lifecycle

A transaction begins with user input and proceeds through authentication, field validation, normalization, persistence, metric recalculation, and dashboard refresh. Each stage has a distinct responsibility. If validation fails, the record should not be stored. If persistence fails, the interface should report the error without displaying a successful save. This controlled sequence prevents inconsistent ledger totals.

C.3 Monthly Aggregation

The reporting service can group valid transactions by month and category. Total income and total expenses are then computed from the corresponding groups, and the net monthly balance is obtained by subtracting expenses from income. Category percentages provide a second perspective on spending. The system should distinguish a month with no records from a month in which a valid recorded total is zero.

C.4 Data Quality Considerations

Duplicate entries, wrong categories, incorrect dates, and inconsistent units can distort every downstream recommendation. The interface should allow users to review and correct records while retaining appropriate access controls. Where edits affect a prior month, the relevant aggregates and derived dashboard indicators should be recalculated. Backups and restore testing are also important in a production implementation.

ANNEXURE D: AI RECOMMENDATION DESIGN

D.1 Hybrid Decision Approach

The project describes a hybrid pipeline in which deterministic financial calculations precede heuristic checks and persona classification. This ordering is useful because recommendations should be based on validated, reproducible indicators. Rules can flag conditions such as negative cash flow or elevated debt burden before a classification model considers the broader profile. The output remains educational guidance rather than an automated financial decision.

D.2 Feature Preparation

Candidate features include savings rate, debt-to-income ratio, emergency runway, discretionary spending share, goal horizon, and stated risk tolerance. Each feature needs a documented definition and consistent unit. Missing inputs should not silently be treated as reliable observations. Features derived from the same income or expense totals should be checked for consistency so that the explanation matches the dashboard.

D.3 Explainability

An explainable output identifies the measured indicator, the rule or decision path used, and the reason a particular suggestion was presented. For example, a recommendation about building a reserve should identify the estimated runway and the comparison threshold. The user should be able to distinguish an observed financial calculation from a model-generated classification and from an illustrative planning assumption.

D.4 Model Limitations

Representative synthetic profiles can help prototype classification behavior but do not establish real-world performance across different households. Evaluation should check class balance, precision, recall, and errors for each persona. Thresholds and training assumptions require review before any real deployment. Recommendations should avoid claiming certainty about future returns or personal suitability.

ANNEXURE E: SECURITY AND RESPONSIBLE DESIGN

E.1 Confidentiality and Access Control

Financial records are sensitive and should be available only to their authorized owner. Authentication confirms identity; authorization verifies that a requested transaction or goal belongs to the authenticated account. Passwords should be stored as salted hashes rather than plaintext. Sensitive data should not appear in application logs, error pages, or unnecessary exports.

E.2 Secure Application Practices

HTTPS protects data in transit. Parameterized database queries help prevent injection, while server-side validation limits malformed inputs. Session handling should use appropriate expiration and protected cookies. Dependencies should be maintained, error messages should avoid disclosing internal implementation details, and backups should be protected with access controls.

E.3 Privacy by Design

Collect only information required for the stated planning functions. A clear notice should explain what is stored, why it is used, and how users can correct or remove records. Future integrations with bank-data services should require informed permission and careful assessment of data retention. Demo screenshots and academic presentations should use sample or appropriately masked data.

E.4 Responsible Recommendation Practices

The application should present its guidance as educational information and display underlying assumptions. It should avoid concealing uncertainty, treating a classification as a fixed personal trait, or encouraging users to make high-stakes decisions solely on an automated suggestion. Clear explanations and opportunities to revise inaccurate inputs support user understanding.

ANNEXURE F: TESTING AND QUALITY ASSURANCE

F.1 Unit Testing

Unit tests should verify the individual formulas for net balance, savings rate, debt-to-income ratio, emergency runway, and required monthly goal contribution. Include typical, boundary, zero-denominator, and invalid-input cases. Tests should check expected numeric precision and confirm that the displayed values match the calculation service.

F.2 Integration Testing

Integration tests should verify that validated form submissions reach the correct database tables and that dashboard aggregates update after creation, editing, or deletion. Authentication and record ownership checks should be tested across API routes. Error scenarios should confirm that failed writes do not produce misleading success messages or partial visible updates.

F.3 Interface and Acceptance Testing

A usable interface should display clear field errors, readable tables, consistent currency formatting, and meaningful empty states. Testing should cover different screen widths, keyboard navigation, and ordinary user workflows from registration through entry and review. Acceptance scenarios can confirm that a user can understand the reason for each displayed recommendation.

F.4 Evaluation and Reporting

The original report includes a test matrix and illustrative classification metrics. Supplementary quality reporting should document test data, expected outputs, actual outputs, pass/fail criteria, and unresolved limitations. A prototype should distinguish completed verification from tests proposed for future deployment. Regression tests should run after changes to formulas, schemas, or recommendation rules.

ANNEXURE G: DEPLOYMENT, LEARNING AND FUTURE SCOPE

G.1 Deployment Considerations

A deployment environment should separate development settings from production settings, manage secrets outside source code, and configure the database with restricted permissions. Logging and monitoring can help identify failed requests and performance problems without exposing private financial details. A documented backup and restore process supports continuity.

G.2 Maintainability and Documentation

The project benefits from a clear repository structure, API documentation, schema definitions, version control, and a repeatable test suite. Naming conventions and small modules make it easier to update calculations independently of the user interface. Documenting formula definitions and recommendation thresholds is especially important because users depend on consistent explanations.

G.3 Internship Learning Outcomes

The project connects classroom knowledge to practical requirements analysis, interface design, relational modeling, secure application development, numerical validation, and explainable AI. Preparing an end-to-end report also develops technical communication skills: design choices must be stated clearly, figures labeled consistently, and limitations distinguished from implemented features.

G.4 Potential Extensions and Closing Note

Future work may explore consent-based bank-data integration, bilingual explanations, improved transaction categorization, accessibility refinements, and more realistic scenario analysis. Each extension should be evaluated for accuracy, privacy, and user benefit. These supplementary annexures extend the theoretical discussion of the existing AI-Based BANKING SYSTEM without changing its original project scope or asserting that proposed enhancements are already implemented.